

ASSESSMENT TOOL 1 -- PART 1

Code 1

Markov Decision Process Demonstration

```
grid_size <- 4

start <- c(1,1)
goal <- c(4,4)

state <- start

path <- matrix("",grid_size,grid_size)

path[start[1],start[2]] <- "S"
path[goal[1],goal[2]] <- "G"

print("Initial Grid")
print(path)

step <- 1

cat("\nStarting Simulation\n\n")

while(!(all(state==goal))){

  row <- state[1]
  col <- state[2]

  possible <- list()

  if(row>1)
    possible$Up <- c(row-1,col)

  if(row<grid_size)
    possible$Down <- c(row+1,col)

  if(col>1)
    possible$Left <- c(row,col-1)

  if(col<grid_size)
    possible$Right <- c(row,col+1)

  best_action <- NULL
  best_distance <- 100

  for(a in names(possible)){

    next_state <- possible[[a]]

    distance <- abs(goal[1]-next_state[1]) +
```

```

    abs(goal[2]-next_state[2])

    if(distance < best_distance){

        best_distance <- distance
        best_action <- a
        state_next <- next_state

    }

}

reward <- -1

if(all(state_next==goal))
    reward <- 100

cat("Step:",step,"\n")

cat("Current State :",state,"\n")
cat("Action Taken :",best_action,"\n")
cat("Next State   :",state_next,"\n")
cat("Reward      :",reward,"\n\n")

state <- state_next

step <- step+1

}

```

OUTPUT:

```
[1] "Initial Grid"
      [,1] [,2] [,3] [,4]
[1,] "S"  ""  ""  ""
[2,] ""   ""  ""  ""
[3,] ""   ""  ""  ""
[4,] ""   ""  ""  "G"
```

Starting Simulation

Step: 1

```
Current State : 1 1
Action Taken  : Down
Next State    : 2 1
Reward       : -1
```

Step: 2

```
Current State : 2 1
Action Taken  : Down
Next State    : 3 1
Reward       : -1
```

Step: 3

```
Current State : 3 1
Action Taken  : Down
Next State    : 4 1
Reward       : -1
```

Step: 4

```
Current State : 4 1
Action Taken  : Right
Next State    : 4 2
Reward       : -1
```

Step: 5

```
Current State : 4 2
Action Taken  : Right
Next State    : 4 3
Reward       : -1
```

Code 2:

R Program to Visualize an Agent's Path in a 4×4 Grid World

```
grid_size <- 4

x <- c(1,1,2,3,4,4,4)
y <- c(1,2,2,2,2,3,4)

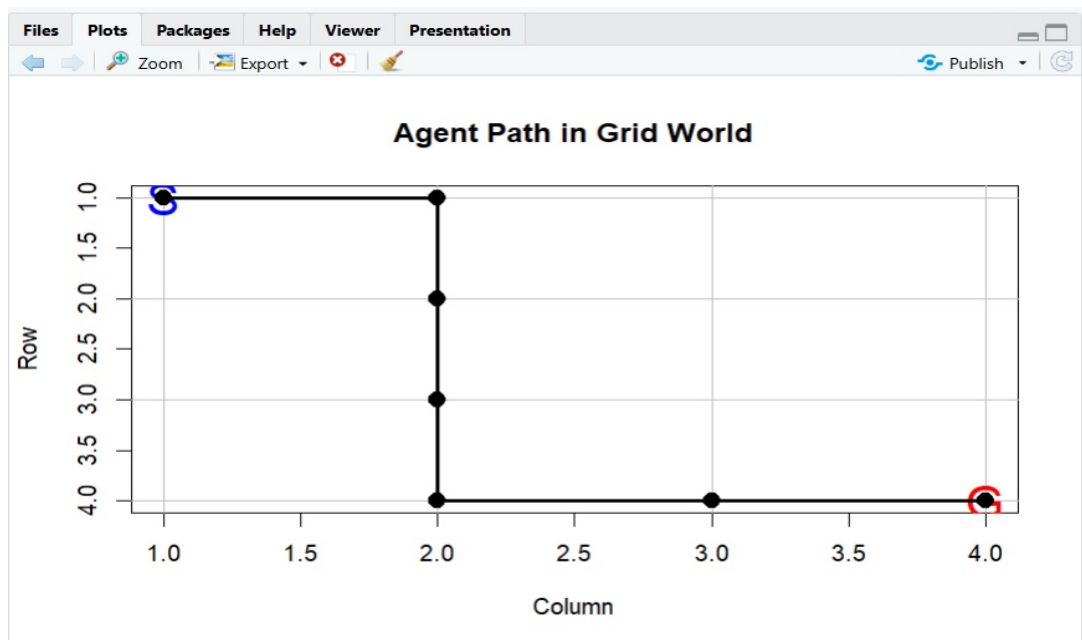
plot(
  1:grid_size,
  1:grid_size,
  type = "n",
  xlim = c(1,4),
  ylim = c(4,1),
  xlab = "Column",
  ylab = "Row",
  main = "Agent Path in Grid World"
)

abline(v = 1:4, col = "gray")
abline(h = 1:4, col = "gray")

text(1,1,"S",cex=2,col="blue")
text(4,4,"G",cex=2,col="red")

lines(y,x,lwd=3)
points(y,x,pch=19,cex=1.5)
```

OUTPUT:



Code 3:

#R Program to Visualize the Markov Decision Process (MDP) Workflow Using DiagrammeR

```
library(DiagrammeR)

grViz("
digraph MDP {

graph[
layout=dot
rankdir=TB
splines=polyline
nodesep=0.6
ranksep=0.9
]

node[
shape=box
style='filled,rounded'
fontname='Times New Roman'
fontsize=24
penwidth=2
color='#555555'
]

A [label='Markov Decision Process',
fillcolor='#1565C0',
fontcolor='white']

B [label='State Space (S)',
fillcolor='#90CAF9']

C [label='Action Space (A)',
fillcolor='#A5D6A7']

D [label='Transition Model\nP(s'|s,a)',
fillcolor='#FFE082']

E [label='Reward Function\nR(s,a)',
fillcolor='#FFCC80']

F [label='Policy\nπ(s)',
fillcolor='#CE93D8']

G [label='Value Function\nV(s)',
fillcolor='#B39DDB']

H [label='Optimal Action',
fillcolor='#4DB6AC',
fontcolor='white']

I [label='Maximum Expected Reward',
shape=ellipse,
fillcolor='#E53935',
fontcolor='white']
```

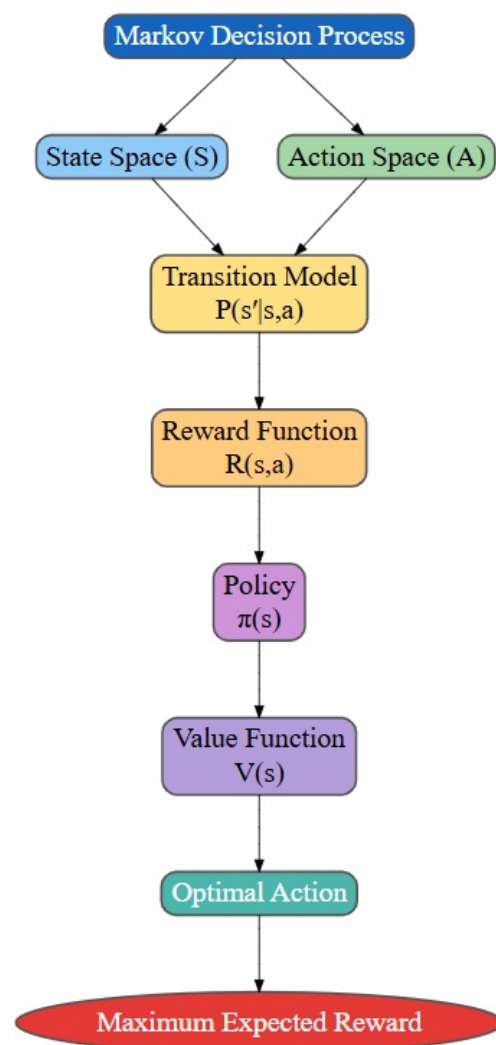
A->B
A->C

B->D
C->D

D->E
E->F
F->G
G->H
H->I

}
")

OUTPUT :



Code 4:

Input data

```
x <- matrix(c(0,1,2,3,4,5,6,7,8,9), ncol = 1)
```

Output labels

```
y <- c(0,1,0,1,0,1,0,1,0,1)
```

Build model

```
model <- keras_model_sequential(input_shape = c(1)) |> layer_dense(units = 8, activation = "relu") |>  
layer_dense(units = 1, activation = "sigmoid")
```

Compile

```
model |> compile( optimizer = "adam", loss = "binary_crossentropy", metrics = "accuracy" )
```

Train

```
model |> fit( x, y, epochs = 300, verbose = 0 )
```

Predict

```
pred12 <- predict(model, matrix(12, ncol = 1)) pred15 <- predict(model, matrix(15, ncol = 1))
```

```
cat("Prediction for 12:", pred12, "\n") cat("Prediction for 15:", pred15, "\n")
```

OUTPUT:

Data	
pred12	num [1, 1] 0.654
pred15	num [1, 1] 0.706
x	num [1:10, 1] 0 1 2 3 4 5 6 7 8 9
Values	
model	<Sequential name=sequential, built=True>
y	num [1:10] 0 1 0 1 0 1 0 1 0 1

CODE 5

Disease Prediction using Neural Network

```
library(keras3) library(tensorflow) library(caret) library(ggplot2)
```

Create Synthetic Disease Data

```
set.seed(42)

n <- 1000

Age <- sample(20:79, n, replace = TRUE) BloodPressure <- sample(90:179, n, replace = TRUE) Sugar
<- sample(70:219, n, replace = TRUE) Cholesterol <- sample(120:319, n, replace = TRUE)

Disease <- ifelse( (BloodPressure > 140 & Sugar > 140) | (Cholesterol > 240), 1, 0 )

data <- data.frame( Age, BloodPressure, Sugar, Cholesterol, Disease )

cat("First 6 Records\n") print(head(data))
```

Prepare Data

```
X <- data[,1:4] y <- data$Disease

set.seed(42)

trainIndex <- createDataPartition( y, p = 0.80, list = FALSE )
```



```
X_train <- X[trainIndex, ] X_test <- X[-trainIndex, ]
```

```
y_train <- y[trainIndex] y_test <- y[-trainIndex]
```

Standardize Data

```
train_mean <- apply(X_train,2,mean) train_sd <- apply(X_train,2,sd)
```

```
X_train <- scale( X_train, center = train_mean, scale = train_sd )
```

```
X_test <- scale( X_test, center = train_mean, scale = train_sd )
```

Build Neural Network

```
model <- keras_model_sequential()
```

```
model |> layer_dense( units = 16, activation = "relu", input_shape = c(4) ) |> layer_dense( units = 8,  
activation = "relu" ) |> layer_dense( units = 1, activation = "sigmoid" )
```

Compile Model

```
model |> compile(
```

```
optimizer = "adam",
```

```
loss = "binary_crossentropy",
```

```
metrics = list("accuracy")
```

```
)
```

Train Model

```
history <- model |> fit(
```

```
x = as.matrix(X_train),
```

```
y = y_train,
```

```
epochs = 30,
```

```
batch_size = 32,
```

```
validation_split = 0.20,
```

```
verbose = 1
```

```
)
```

Evaluate Model

```
score <- model |> evaluate(  
  as.matrix(X_test),  
  y_test,  
  verbose = 0  
)  
  
cat("\n") cat("Test Accuracy :", score[[2]], "\n")
```

Predict One Patient

```
sample <- matrix( c(50,160,180,260), nrow = 1 )  
  
sample <- scale(  
  sample,  
  center = train_mean,  
  scale = train_sd  
)  
  
prediction <- predict( model, sample )  
  
cat("\nPrediction Probability :", prediction[1,1], "\n")  
  
if(prediction[1,1] > 0.5) { cat("Disease Detected\n") } else { cat("Healthy\n") }
```

Plot Accuracy

```
accuracy_df <- data.frame(  
  Epoch = 1:30,  
  Training = history$metrics$accuracy,  
  Validation = history$metrics$val_accuracy  
)  
  
ggplot(  
  accuracy_df,  
  aes(EPOCH)
```

```

) +

geom_line(

aes( y = Training, colour = "Training Accuracy" ),

linewidth = 1

) +

geom_line(

aes( y = Validation, colour = "Validation Accuracy" ),

linewidth = 1

) +

labs(

title = "Model Accuracy",

y = "Accuracy",

colour = ""

) +

theme_minimal()

```

Plot Loss

```

loss_df <- data.frame(

Epoch = 1:30,

Training = history$metrics$loss,

Validation = history$metrics$val_loss

)

ggplot(

loss_df,

aes(EPOCH)

) +

geom_line(

aes( y = Training, colour = "Training Loss" ),

linewidth = 1

```

```

)+

geom_line(

aes(y = Validation, colour = "Validation Loss" ),

linewidth = 1

)+

labs(

title = "Model Loss",

y = "Loss",

colour = ""

)+

theme_minimal()

```

OUTPUT:

